WEEK 1 — DEVOPS & GIT

DevOps Mindset & Advanced Git Workflow

Culture • Git Internals • Version Control

Sami Selim

Instructor · DevOps Engineering

SECTION 1 — AGENDA

DevOps Mindset & Advanced Git Workflow

From Git foundations to branching strategies, internals, and recovery.

01

VERSION CONTROL

Local, centralized, distributed

02

GIT ARCHITECTURE

Objects, SHA-1, three trees

03

DEVOPS CULTURE

CALMS, collaboration, flow

04

BRANCHING & REBASE

GitFlow, trunk-based, cherry-pick

05

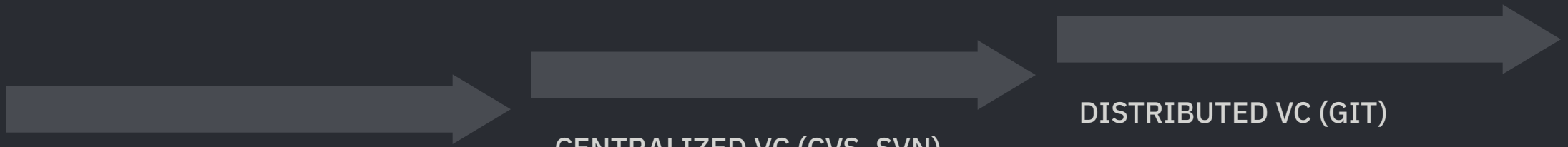
GIT COMMANDS

Essential commands: setup, staging, branching, history, recovery

SECTION 1 — VERSION CONTROL

Introduction to Version Control

From local file copies to distributed repositories, each generation solved a critical weakness in the last.



Clone vs. Download

Clone

- Full repo copy
- Entire history
- Can push/pull

Download

- Just files
- No history
- No Git connection

SECTION 1 — ORIGINS

The Origins of Git

Necessity is the mother of invention.

2005
CREATED

C + Rust
LANGUAGE

90%+
MARKET SHARE

📝 Instructor Notes: See handout — Git Origins & Version Control Types

SECTION 2 — ARCHITECTURE

Why Git? Architecture & Internals

Working Directory

Your files live here: **a.txt**, **src/**, and other local changes.

- Untracked and modified files appear here first
- Represents the current state of your workspace
- Changes are not saved to Git until you commit

.git Repository

This hidden folder is the Git repository.

- Stores the objects database
- Contains blobs, trees, and commits
- Preserves full history of every version

BLOB

Stores file content
No filename

TREE

Stores directory structure
References blobs

COMMIT

Snapshot pointer
Author, message, parent

Hashing Example

```
echo "Hello" | git hash-object --stdin
→ SHA-1 hash
```

SECTION 2 — WHY GIT?

What Git Was Designed to Do

Four core design goals that make Git different from every other VCS.

TRACKING EVERYTHING

Git tracks every change to every file — content, renames, deletions, and permissions. Nothing is ever silently lost. Even empty folders and file modes are recorded.

OS INDEPENDENT

Git runs identically on Linux, macOS, and Windows. A repo created on any OS works on all others. No platform-specific line endings or path issues — Git handles it all.

UNIQUE ID (SHA-1)

Every commit, file, and tree is identified by a unique 40-character SHA-1 hash. This makes it impossible to silently corrupt data — any change produces a completely different hash.

TRACKING HISTORY

Git stores the full history of every branch, merge, and revert. You can travel back to any point in time with git checkout, inspect changes with git log, and recover anything with git reflog.

SECTION 2 — OBJECT DATABASE

Git's Object Model

Git is a **content-addressable file system**. Every object is identified by a SHA-1 hash — even a 1-character change produces a completely different hash.

01

BLOB

Raw file content. No filename stored. Pure data.

02

TREE

Maps filenames to blobs. Represents a directory.

03

COMMIT

Points to a tree + parent commit + author + timestamp

04

TAG

Named pointer to a specific commit

SHA-1 in Practice

```
echo "Hello" | git hash-object --stdin
echo "Hello" | shasum
```

Key Properties

- Deterministic — same input = same hash
- Avalanche effect — 1 bit change = totally different hash
- Collision-resistant — virtually impossible to duplicate

SECTION 2 — GIT ARCHITECTURE

The Three Trees of Git

Know where changes live — and how they move.

01

Working Directory

- Your editable files
- Modified or untracked

02

Staging Area

- Curated next commit
- `git add` and `git add -p`

03

Local Repository

- Committed snapshots
- `.git` object database

Flow: `edit → stage → commit`

SECTION 3 — DEVOPS

Introduction to DevOps

The Wall of Confusion

Dev vs. Ops

- **Dev:** ship fast
- **Ops:** stay stable
- **Result:** friction

DevOps

- **People** + process + products
- Shared delivery ownership
- Continuous delivery end to end
- Speed without losing stability

SECTION 3 — DEVOPS

Life Without DevOps

The traditional software delivery model — and why it breaks down.

🏢 Traditional Team Structure

- Dev team writes code in isolation for weeks/months
- Code "thrown over the wall" to Ops team
- Ops team sees the code for the first time on release day
- Manual deployments — SSH into servers, copy files
- No shared responsibility — "It works on my machine"
- Rollback = panic + all-hands incident call

📉 The Result

- Release cycles: 3–6 months
- Deployment day = high-risk event (war room)
- Bugs found in production (not in testing)
- Finger-pointing between Dev and Ops
- Slow feedback loop — users wait months for fixes
- High MTTR (Mean Time To Recovery): hours or days

SLOW RELEASES

Netflix in 2008: manual deployments took days. One bad deploy could take down the entire monolith.

FRAGILE DEPLOYMENTS

Amazon pre-2010: deployments were so risky they were done at 3am with the whole team on standby.

SECTION 4 — BRANCH STRATEGIES

GitFlow — Structured Release Model

Two opposing models for branch management. GitFlow suits scheduled, versioned releases.

Branch Structure

- `main` — production-ready code only. Tagged with version numbers (`v1.0`, `v2.0`)
- `develop` — integration branch. All features merge here first
- `feature/*` — branch from `develop`. One branch per feature/ticket
- `release/*` — branch from `develop` when ready to ship. Bug fixes only
- `hotfix/*` — branch from `main`. Emergency production fixes only

Workflow Steps

1. Create a `feature/*` branch from `develop`
2. Build and test the feature in isolation
3. Merge the feature into `develop`
4. Cut a `release/*` branch from `develop`
5. Apply bug fixes only on the release branch
6. Merge the release branch into `main` and tag the version
7. Back-merge release fixes into `develop` and ship to production

BEST FOR

Teams that plan releases in advance and need strong version control, release stabilization, and structured approvals.

DRAWBACKS

More branching overhead, slower integration, and higher merge complexity than trunk-based development.

SECTION 4 — BRANCH STRATEGIES

Trunk-Based Development — Continuous Integration Model

Every developer integrates to main at least once a day. No long-lived branches. Always deployable.

Branch Structure

- **main** (trunk) — the ONLY long-lived branch. Always deployable
- `feature/*` — short-lived (max 1–2 days). Merged back to main ASAP
- Feature Flags — incomplete features hidden behind flags, not branches
- Release tags — cut directly from main when needed

Workflow Steps

1. Pull latest main
2. Create short-lived branch `feature/add-button`
3. Make small, focused commits
4. Open PR → automated CI must pass
5. Merge to main within 24–48 hours (no exceptions)
6. Feature flag hides incomplete work in production
7. Deploy main to production multiple times per day

BEST FOR

SaaS products, web apps, teams doing CI/CD, high-frequency deployments. Google, Facebook, Netflix deploy 100s of times/day using TBD.

DRAWBACKS

Requires mature CI/CD pipeline, discipline with feature flags, harder for junior teams, less suited for versioned/packaged software.

SECTION 4 — BRANCH STRATEGIES

GitFlow vs. Trunk-Based — Side by Side

Choose the right model for your team and release cadence.

Criteria	GitFlow	Trunk-Based
Branch lifespan	Weeks / months	Hours / days
Merge complexity	High — conflicts common	Low — small diffs
Release cadence	Scheduled (weekly/monthly)	Continuous (multiple/day)
CI/CD requirement	Optional	Mandatory
Feature isolation	Separate branches	Feature flags
Best team size	Large, distributed	Any (with CI discipline)
Used by	Enterprise, mobile apps	Google, Netflix, Spotify

CHOOSE GITFLOW IF

You ship versioned software (mobile apps, SDKs, enterprise), have scheduled release windows, or need strict QA gates before production.

CHOOSE TRUNK-BASED IF

You run a SaaS/web product, deploy frequently, have a solid CI/CD pipeline, and want to eliminate merge hell.

SECTION 4 — VERSION CONTROL

Rebase vs. Merge

Two ways to integrate branch changes. One preserves history. One rewrites it.

git merge

- Creates a **merge commit**
- Preserves branch history
- Safe for shared branches

git rebase

- Reapplies commits on top of `main`
- Creates clean, linear history
- Use only on private branches

⊗ **Golden rule:** never rebase shared commits. Rebase local work. Merge shared work.

SECTION 4 — ADVANCED GIT

Advanced Git Operations

Three techniques for precise commit control and safer workflow automation.

Cherry-Pick

- Apply one commit anywhere
- Target hotfixes fast
- Keep unrelated work out

Squash

- Combine WIP commits
- One clean merge commit
- Cleaner history

Hooks

- Run automatically on events
- `pre-commit` and `pre-push`
- Block bad commits early

SECTION 5 — GIT COMMANDS

Git Setup & Configuration

First things first — configure your identity before any commit.

Global Config

```
git config --global user.name "Sami Selim"
git config --global user.email "sami@example.com"
git config --global core.editor "code --wait"
git config --list
```

Initialize a Repo

```
git init                # new local repo
git clone <url>         # clone remote repo
git clone <url> my-folder # clone into folder
```

--global

SCOPE: applies to all repos

--local

SCOPE: applies to this repo only

--system

SCOPE: applies to all users

SECTION 5 — GIT COMMANDS

Staging & Committing

The core Git workflow — track, stage, and snapshot your changes.

01

MODIFY

Edit files in Working Directory

02

STAGE

git add → move to Staging Area (Index)

03

COMMIT

git commit → save snapshot to Repository

04

PUSH

git push → sync to remote

Staging Commands

```
git status                  # show working tree status
git add <file>              # stage a specific file
git add .                   # stage all changes
git add -p                  # stage interactively (hunks)
git restore --staged <file> # unstage a file
```

Commit Commands

```
git commit -m "feat: add login"  # commit with message
git commit --amend               # edit last commit
git commit -am "fix: typo"       # stage+commit tracked files
git diff --staged                # review staged changes
```

SECTION 5 — GIT COMMANDS

Branching & Merging

Isolate work, collaborate safely, and integrate changes.

Branch Commands

```
git branch                  # list all branches
git branch feature/login    # create new branch
git switch feature/login    # switch to branch
git switch -c feature/login # create + switch
git branch -d feature/login # delete merged branch
git branch -D feature/login # force delete branch
git branch -m old new       # rename branch
```

Merge Commands

```
git merge feature/login     # merge into current branch
git merge --no-ff feature   # always create merge commit
git merge --squash feature  # squash all commits into one
git merge --abort           # abort a conflicted merge
git diff main..feature      # compare branches
```

FAST-FORWARD MERGE

No divergence. Git just moves the pointer forward. No merge commit created.

THREE-WAY MERGE

Branches diverged. Git creates a new merge commit combining both histories.

SECTION 5 — GIT COMMANDS

Remote Repositories

Connect, sync, and collaborate with remote repos.

Remote Management

```
git remote -v                    # list remotes
git remote add origin <url>      # add remote
git remote remove origin         # remove remote
git remote rename origin upstream  # rename remote
git fetch origin                 # fetch without merge
git fetch --all                  # fetch all remotes
```

Push & Pull

```
git pull origin main             # fetch + merge
git pull --rebase origin main    # fetch + rebase
git push origin main             # push to remote
git push -u origin feature/login # push + set upstream
git push --force-with-lease      # safe force push
git push origin --delete feature/old  # delete remote branch
```

FETCH — Downloads changes but does NOT merge. Safe to run anytime.

PULL — fetch + merge (or rebase). Updates your local branch.

PUSH — Uploads your local commits to the remote. Requires up-to-date branch.

SECTION 5 — GIT COMMANDS

History & Inspection

Understand what happened, when, and why.

Log & Diff

```
git log                    # full commit history
git log --oneline          # compact one-line view
git log --oneline --graph  # visual branch graph
git log --author="Sami"    # filter by author
git log -p                 # show patch/diff per commit
git log --since="2 weeks ago"  # filter by date
git diff                   # unstaged changes
git diff HEAD              # all changes vs last commit
git diff branch1..branch2  # compare two branches
```

Inspect & Search

```
git show <commit>          # show commit details
git show HEAD~2            # show 2 commits back
git blame <file>           # who changed each line
git grep "TODO"            # search in tracked files
git shortlog -sn           # commits per author
git log --all --oneline    # all branches history
```

HEAD — A pointer to the current commit (tip of current branch). Moving HEAD moves your view of the repo.

HEAD~N — Refers to N commits before HEAD. e.g. HEAD~1 = parent, HEAD~3 = 3 commits back.

SECTION 5 — GIT COMMANDS

Undoing & Recovery

Git almost never loses data — if you know the right commands.

RESTORE

Undo file changes (safe, doesn't touch commits)

RESET

Move HEAD backward (rewrites local history)

REVERT

Create a new commit that undoes a past commit (safe for shared branches)

Undo Changes

```
git restore <file>          # discard working dir changes
git restore --staged <file>     # unstage file
git clean -fd               # remove untracked files/dirs
git revert <commit>          # safe undo (new commit)
git revert HEAD             # revert last commit
```

Reset (Use with Caution)

```
git reset --soft HEAD~1       # undo commit, keep staged
git reset --mixed HEAD~1       # undo commit, keep unstaged
git reset --hard HEAD~1       # undo commit, discard changes
git reflog                 # view all HEAD movements
git checkout <hash>          # inspect old commit (detached)
```

SECTION 5 — GIT COMMANDS

Stash, Tags & .gitignore

Essential utilities for cleaner, more organized workflows.

Stash

```
git stash                    # stash current changes
git stash push -m "WIP login"    # stash with a name
git stash list               # list all stashes
git stash pop                # apply + remove top stash
git stash apply stash@{2}    # apply specific stash
git stash drop stash@{0}     # delete a stash
git stash clear              # delete all stashes
```

Tags

```
git tag                      # list all tags
git tag v1.0.0               # lightweight tag
git tag -a v1.0.0 -m "Release"   # annotated tag
git push origin v1.0.0       # push a tag
git push origin --tags       # push all tags
git tag -d v1.0.0            # delete local tag
```

.gitignore

```
# Ignore node_modules
node_modules/

# Ignore build output
dist/
*.log
*.env

# Ignore OS files
.DS_Store
Thumbs.db

# Ignore IDE files
.vscode/
.idea/
```

SECTION 6 — SUMMARY

Best Practices

→

Commit early

Local checkpoints

→

Use semantic messages

feat:, fix:, docs:, chore:

→

Protect shared history

Rebase private. Merge public.

→

Shift left on quality

Lint, test, scan before CI

SECTION 6 — GITHUB ACTIONS

What is GitHub Actions?

Native CI/CD automation built directly into your GitHub repository.

CI (Continuous Integration)

Automatically build and test code on every push or pull request

CD (Continuous Delivery)

Automatically deploy tested code to staging or production

Automation

Run any script triggered by any GitHub event (push, PR, schedule, issue, etc.)

Why GitHub Actions?

- No external CI server needed
- Free for public repos
- 2,000 free minutes/month (private)
- Marketplace with 20,000+ actions
- Native GitHub integration (secrets, PRs, issues)
- Matrix builds across OS & versions

Key Concepts

- Workflow — Automated process defined in YAML
- Event — Trigger that starts a workflow
- Job — A set of steps that run on a runner
- Step — Individual task (run command or use action)
- Runner — Virtual machine that executes jobs
- Action — Reusable unit of automation

SECTION 6 — GITHUB ACTIONS

Workflow Architecture

How all the pieces connect — from trigger to deployment.

01

EVENT

A trigger fires (push, pull_request, schedule, workflow_dispatch)

02

WORKFLOW

GitHub reads .github/workflows/*.yml file

03

JOB

One or more jobs are queued (can run in parallel or sequentially)

04

RUNNER

GitHub spins up a virtual machine (ubuntu-latest, windows-latest, macos-latest)

05

STEPS

Each step runs a shell command or a pre-built Action

06

RESULT

Pass ✓ or Fail ✗ reported back to the commit/PR

Workflow File Location

```
.github/
  workflows/
    ci.yml          ← runs on push
    deploy.yml      ← runs on release
    nightly.yml     ← runs on schedule
```

Common Event Triggers

```
on: push                # any push
on: pull_request        # PR opened/updated
on: release             # new release published
on: schedule            # cron job
  cron: '0 2 * * *'     # every day at 2am
on: workflow_dispatch   # manual trigger
on: issues              # issue opened/closed
```

SECTION 6 — GITHUB ACTIONS

Writing Your First Workflow

A complete annotated CI workflow — line by line.

ci.yml — Basic CI Pipeline

```yaml
name: CI Pipeline                # workflow name (shown in GitHub UI)

on:                              # TRIGGERS
  push:
    branches: [main, develop]    # run on push to these branches
  pull_request:
    branches: [main]             # run on PRs targeting main

jobs:                            # JOBS
  build-and-test:                # job ID (can be anything)
    runs-on: ubuntu-latest       # runner OS

    steps:
      - name: Checkout code      # STEP 1 — get the code
        uses: actions/checkout@v4

      - name: Setup Node.js      # STEP 2 — install runtime
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies  # STEP 3 — install packages
        run: npm ci

      - name: Run linter         # STEP 4 — lint
        run: npm run lint

      - name: Run tests          # STEP 5 — test
        run: npm test

      - name: Build              # STEP 6 — build
        run: npm run build
```

SECTION 6 — GITHUB ACTIONS

Workflow Syntax — Key Keywords

The three building blocks of every step in a workflow.

uses:

— Runs a pre-built Action from GitHub Marketplace (e.g. actions/checkout@v4). Always pin to a version tag.

run:

— Executes a raw shell command directly on the runner. Supports multi-line with the pipe operator.

with:

— Passes named input parameters to an Action (e.g. node-version: 20, cache: npm).

Single-line vs Multi-line run:

```
run: npm test

run: |
  echo "Starting build..."
  npm ci
  npm run build
  echo "Done!"
```

Pinning Actions for Security

```
# Unsafe - floating tag
uses: actions/checkout@main

# Good - version tag
uses: actions/checkout@v4

# Best - full commit SHA
uses:
actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
```

SECTION 6 — GITHUB ACTIONS

Secrets, Environment Variables & Matrix Builds

Secure configuration and multi-environment testing.

Secrets & Env Variables

```yaml
# Define in: GitHub → Settings → Secrets → Actions

env:                          # workflow-level env var
  NODE_ENV: production

jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      APP_URL: ${{ vars.APP_URL }}    # repo variable (not secret)
    steps:
      - name: Deploy
        run: ./deploy.sh
        env:
          API_KEY: ${{ secrets.API_KEY }}        # secret
          DB_PASS: ${{ secrets.DB_PASSWORD }}    # secret
          TOKEN:   ${{ secrets.GITHUB_TOKEN }}   # auto-provided
```

Matrix Builds

```yaml
# Test across multiple versions/OS at once

jobs:
  test:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        node: [18, 20, 22]
      fail-fast: false            # don't cancel all on one fail
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm test
# Result: 6 parallel jobs (2 OS × 3 Node versions)
```

secrets.* — Encrypted values. Never visible in logs. Set in repo/org Settings. Use for API keys, tokens, passwords.

vars.* — Plain text config values. Visible in logs. Use for non-sensitive config like URLs, feature flags.

SECTION 6 — GITHUB ACTIONS

CI/CD Pipeline — Part 1: Test & Build

Trigger the pipeline, run tests, build and push a Docker image.

deploy.yml — Full CI/CD Pipeline

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [main]

jobs:
  # ——— JOB 1: Test ———————————————————————————————
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm test

  # ——— JOB 2: Build & Push Docker Image ——————————————
  build:
    needs: test                    # only runs if test passes
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USER }}
          password: ${{ secrets.DOCKER_TOKEN }}
      - name: Build & Push
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: myapp:${{ github.sha }}
```

SECTION 6 — GITHUB ACTIONS

CI/CD Pipeline — Part 2: Deploy to Production

Gate the deployment with environment protection and deploy via SSH.

deploy.yml — Job 3: Deploy

```yaml
# Job 3: Deploy to Production
deploy:
  needs: build                    # only runs if build passes
  runs-on: ubuntu-latest
  environment: production         # requires manual approval
  steps:
    - name: Deploy via SSH
      uses: appleboy/ssh-action@v1
      with:
        host: ${{ secrets.SERVER_HOST }}
        username: ${{ secrets.SERVER_USER }}
        key: ${{ secrets.SSH_KEY }}
        script: |
          docker pull myapp:${{ github.sha }}
          docker stop myapp || true
          docker run -d --name myapp -p 80:3000 myapp:${{ github.sha }}
```

needs: — Job dependency. This job only runs if the referenced job succeeds. Chains jobs in sequence.

environment: production — Triggers protection rules. Can require manual reviewer approval before running.

github.sha — Built-in context variable. The full commit SHA that triggered the workflow. Used to tag Docker images uniquely.

SECTION 6 — GITHUB ACTIONS

GitHub Actions Best Practices

Write workflows that are fast, secure, and maintainable.

Pin Action Versions — Always use @v4 or a full SHA, never @main. Prevents supply chain attacks.

Cache Dependencies — Use actions/cache or built-in cache: 'npm' to speed up installs by 60–80%.

Least Privilege — Set permissions: read-all at top, then grant only what each job needs.

Fail Fast — Use fail-fast: true in matrix builds to cancel remaining jobs on first failure.

Use Environments — Gate production deploys with environment: production and required reviewers.

Keep Secrets Secret — Never echo secrets. Use ${{ secrets.X }} — GitHub auto-masks them in logs.

SECTION 6 — GITHUB ACTIONS

Permissions & Built-in Contexts

Lock down your workflows and use GitHub's built-in variables.

Permissions Best Practice

```yaml
permissions:
  contents: read      # default: read-only

jobs:
  deploy:
    permissions:
      contents: write  # only this job needs write
      id-token: write  # for OIDC auth (no long-lived secrets)
```

Useful Built-in Contexts

```
${{ github.actor }}      # user who triggered the run
${{ github.ref }}        # branch or tag ref
${{ github.sha }}        # full commit SHA
${{ github.repository }} # owner/repo
${{ runner.os }}         # Linux / Windows / macOS
${{ job.status }}        # success / failure / cancelled
```

CONCLUSION

What We Covered Today

From version control fundamentals to production-grade CI/CD automation.

Version Control

Local → Centralized → Distributed. Git as the industry standard since 2005.

Git Internals

Blobs, Trees, Commits, SHA-1 hashing, the .git object database.

DevOps Mindset

Breaking the wall of confusion. Culture, collaboration, and automation.

Branch Strategies

GitFlow for scheduled releases. Trunk-Based for continuous delivery.

Git Commands

Setup, staging, branching, remotes, history, recovery, stash, and tags.

GitHub Actions

Workflows, triggers, jobs, secrets, matrix builds, and full CI/CD pipelines.

Key Takeaways

- Git is not just a tool — it's a communication protocol for teams
- Small, frequent commits beat large, infrequent ones every time
- Never rewrite shared history — use revert on public branches
- Automate everything repeatable — if you do it twice, script it
- Your CI/CD pipeline is as important as your application code

What's Next — Week 2

- Docker & Containerization
- Writing Dockerfiles and docker-compose
- Container registries and image management
- Kubernetes fundamentals
- Deploying containerized apps to the cloud

> **"Any sufficiently advanced automation is indistinguishable from magic — until you understand the YAML."**
> **— DevOps Proverb**

WEEK 1 — DEVOPS & GIT

Thank You

Questions, feedback, or just want to talk Git?

Connect

- Instructor: Sami Selim
- Role: DevOps Engineering Instructor
- Email: samiselim75@gmail.com
- GitHub: github.com/samiselim

Resources

- Pro Git Book
- GitHub Docs
- GitHub Actions Marketplace

> **"The best time to learn Git properly was when you started coding. The second best time is now."**
> **— DevOps Proverb**